Teach Module 10: GitHub Copilot Fundamentals for Java Developers

Module 10 focuses on using GitHub Copilot as a Java developer. The goal is not to let Copilot replace your thinking. The goal is to use it as a fast assistant for drafts, boilerplate, explanations, and ideas while you remain responsible for correctness, design, security, readability, and maintainability.

Learning Goals

By the end of this module, you should be able to:

- Explain what GitHub Copilot is useful for in Java development.
- Write clear prompts that produce better Java code suggestions.
- Use Copilot inside an IDE workflow such as IntelliJ IDEA.
- Review AI-generated code before accepting it.
- Identify common Copilot mistakes.
- Practice a prompt, inspect, edit, verify workflow.

1. What GitHub Copilot Is

GitHub Copilot is an AI coding assistant. For Java developers, think of it as a fast pair programmer that can help you:

- Write boilerplate code
- Generate simple Java classes
- Draft Spring services and controllers
- Suggest repository methods
- Explain unfamiliar code
- Create starter test cases
- Suggest refactoring options

Copilot is helpful, but it is not a senior engineer. It may produce code that looks correct but has design problems, missing edge cases, incorrect assumptions, or security issues.

The best mindset is:

> *Copilot writes suggestions. You make engineering decisions.*

2. What Copilot Is Good At

Copilot is strongest with repetitive and pattern-based work.

Example:

```
public class Customer {
    private Long id;
    private String name;
    private String email;

    // Copilot can quickly suggest constructors, getters, setters, and toString.
}
```

Good Copilot use cases include:

- DTOs
- Entities
- Simple service skeletons
- REST controller drafts
- Java Stream examples
- Basic utility methods
- JUnit test drafts
- Code explanations

However, generated code should always be reviewed.

3. Prompting Copilot Effectively

Copilot works better when you give it clear context.

Weak prompt:

```
make service
```

Better prompt:

```
Create a Spring Boot service class named CustomerService.
It should use CustomerRepository.
Add methods to create a customer, find a customer by id, and list all customers.
Throw CustomerNotFoundException when an id is missing.
```

A strong prompt usually includes:

- The class or method name
- The framework being used
- The expected behavior
- Error handling rules
- Return types
- Constraints or things to avoid

Example prompt as a code comment:

```
// Create a method that validates an email address.
// Return true only if it contains one @ symbol and a domain after the dot.
// Do not use external libraries.
public boolean isValidEmail(String email) {
}
```

4. IntelliJ Workflow

A practical Copilot workflow in IntelliJ:

1. Write the class or method name yourself.
2. Add a short comment describing the intent.
3. Let Copilot suggest code.

4. Read the generated code carefully.
5. Edit it to match your project style.
6. Run or create tests.
7. Refactor if the code is too broad, clever, or messy.

Do not accept suggestions blindly. The final code should be something you understand and can defend in a code review.

5. Example: Service Layer Draft

Copilot might help draft a service like this:

```
@Service
public class ProductService {

    private final ProductRepository productRepository;

    public ProductService(ProductRepository productRepository) {
        this.productRepository = productRepository;
    }

    public Product createProduct(Product product) {
        return productRepository.save(product);
    }

    public Product getProductById(Long id) {
        return productRepository.findById(id)
                .orElseThrow(() -> new ProductNotFoundException(id));
    }

    public List<Product> getAllProducts() {
        return productRepository.findAll();
    }
}
```

After Copilot generates code like this, review it:

- Is `ProductRepository` actually available?
- Does `ProductNotFoundException` exist?
- Should the method return an entity or DTO?
- Should validation happen here?
- Is the service leaking database structure to the controller?
- Are transactions needed?

This review step is where real software engineering happens.

6. Common Copilot Mistakes

Watch for:

- Invented class names or methods
- Incorrect imports

- Missing null checks
- Overly broad exception handling
- Code that compiles but violates the project design
- Security issues
- Tests that only confirm the generated implementation, not the requirement

Risky example:

```
catch (Exception e) {
    return null;
}
```

This is usually poor service design. Prefer meaningful exceptions or controlled failure responses.

7. Copilot Review Checklist

Before accepting Copilot-generated Java code, ask:

- Does it compile?
- Does it match the requirement?
- Does it follow the project package and naming style?
- Are exceptions handled clearly?
- Are edge cases handled?
- Is there duplicated logic?
- Is it secure?
- Is it testable?
- Would I approve this in a code review?

Practice Exercises

Exercise 1: Generate a Plain Java Class

Ask Copilot to create a `Customer` class with:

- `id`
- `name`
- `email`
- Constructors
- Getters and setters
- `toString()`

Review the generated code.

Check:

- Are fields private?
- Are method names correct?
- Is the constructor useful?
- Did Copilot add anything unnecessary?

Exercise 2: Prompt Refinement

Start with a vague prompt:

```
Create product service
```

Then improve it:

```
Create a Spring Boot service named ProductService.
Use constructor injection for ProductRepository.
Add methods to create a product, find a product by id, and list all products.
Throw ProductNotFoundException when a product id is not found.
```

Compare the two outputs.

Goal: learn how better prompts produce better code.

Exercise 3: Generate a Service Class

Create a Spring Boot `OrderService`.

Requirements:

- `placeOrder(Order order)`
- Reject orders with total amount less than or equal to zero
- Save valid orders
- `findOrder(Long id)`
- Throw `OrderNotFoundException` if missing

Use Copilot to draft the service, then edit it yourself.

Exercise 4: Review Copilot Code

Ask Copilot to generate:

```
public boolean isValidEmail(String email)
```

Review the result.

Look for:

- Null handling
- Edge cases
- Overcomplicated regex
- False positives
- Readability

Then improve the method manually.

Exercise 5: Generate Boilerplate for a REST Controller

Prompt Copilot to create a `CustomerController` with endpoints:

- `POST /customers`
- `GET /customers/{id}`
- `GET /customers`

Review:

- Are HTTP status codes appropriate?
- Is the controller too tightly coupled to entities?
- Are request and response types clear?
- Is exception handling missing?

Exercise 6: Explain Existing Code

Paste a small Java method and ask Copilot:

```
Explain what this method does in simple terms.
Identify any possible bugs or edge cases.
```

Use this with code you already understand, so you can judge whether the explanation is accurate.

Exercise 7: Generate Comments, Then Remove Bad Ones

Ask Copilot to add comments to a service class.

Then delete comments that only repeat the code.

Good comment:

```
// Prevents duplicate registrations before creating a new account.
```

Weak comment:

```
// Set the name field
customer.setName(name);
```

Goal: learn that Copilot can over-comment.

Exercise 8: Verification Checklist

For any generated code, complete this checklist:

- Does it compile?
- Does it match the requirement?
- Are names consistent with the project?
- Are exceptions meaningful?
- Are edge cases handled?
- Is the code secure?
- Is the code simple enough?
- Would I approve this in a code review?

Module 10 Lab: Customer Management API

Lab Goal

Use GitHub Copilot to help build a small Spring Boot feature while practicing prompt writing, code review, and responsible acceptance of AI-generated code.

You will build a simple Customer Management API.

Part 1: Create the Domain Model

Create a class named `Customer`.

Fields:

```
private Long id;
private String name;
private String email;
private String phone;
```

Use Copilot to generate:

- Constructors
- Getters and setters
- `toString()`

Prompt idea:

```
Create a Java Customer class with id, name, email, and phone fields.
Add constructors, getters, setters, and toString.
Keep the code simple and readable.
```

Review task:

- Check whether Copilot added unnecessary code such as validation, database annotations, or unrelated fields.

Part 2: Create the Repository

Create an interface named `CustomerRepository`.

If using Spring Data JPA:

```
public interface CustomerRepository extends JpaRepository<Customer, Long> {
}
```

Prompt idea:

```
Create a Spring Data JPA repository for Customer using Long as the ID type.
```

Review task:

- Is the correct entity type used?
- Is the correct ID type used?
- Were unnecessary query methods added?

Part 3: Create the Service

Create a class named `CustomerService`.

Required methods:

```
Customer createCustomer(Customer customer);
Customer findCustomerById(Long id);
List<Customer> findAllCustomers();
void deleteCustomer(Long id);
```

Rules:

- Use constructor injection.
- Throw `CustomerNotFoundException` if a customer is missing.
- Do not return `null`.

Prompt idea:

```
Create a Spring Boot service named CustomerService.
Use constructor injection for CustomerRepository.
Add createCustomer, findCustomerById, findAllCustomers, and deleteCustomer methods.
Throw CustomerNotFoundException when a customer id is not found.
Do not return null.
```

Review task:

- Did Copilot use constructor injection?
- Did it return `null` anywhere?
- Did it invent exception classes?
- Is delete behavior correct?

Part 4: Create the Exception

Create:

```
public class CustomerNotFoundException extends RuntimeException {
}
```

Improve it so it accepts a customer ID:

```
public class CustomerNotFoundException extends RuntimeException {

    public CustomerNotFoundException(Long id) {
        super("Customer not found with id: " + id);
    }
}
```

Prompt idea:

```
Create a RuntimeException named CustomerNotFoundException that accepts a customer id and
creates a clear error message.
```

Part 5: Create the REST Controller

Create `CustomerController`.

Endpoints:

```
POST   /customers
GET    /customers/{id}
GET    /customers
DELETE /customers/{id}
```

Prompt idea:

```
Create a Spring Boot REST controller named CustomerController.
Use CustomerService.
Add endpoints to create a customer, get a customer by id, list all customers, and delete a
customer.
Use proper HTTP methods and ResponseEntity where appropriate.
```

Review task:

- Are endpoint paths correct?
- Are HTTP methods correct?
- Is request body used for create?
- Does delete return a sensible response?
- Is the controller thin, or did Copilot put business logic in it?

Part 6: Test Copilot's Explanation Ability

Pick one generated method and ask Copilot:

```
Explain this method in simple terms.
What edge cases should I test?
```

Then compare the answer with your own judgment.

Part 7: Final Review Checklist

Before calling the lab complete, answer:

- Did I understand every generated line?
- Did I reject or edit any bad suggestion?
- Does the code compile?
- Are names clear and consistent?
- Are exceptions meaningful?
- Did Copilot add unnecessary complexity?
- Is the final code mine in understanding, even if Copilot helped draft it?

Bonus Challenge

Ask Copilot to generate a validation method:

```
private void validateCustomer(Customer customer)
```

Validation rules:

- Name must not be blank.
- Email must contain `@`.
- Phone is optional.

Then manually improve the generated code.

Key Takeaway

The main skill in this module is not typing less. It is:

```
prompt -> inspect -> edit -> verify
```

Use Copilot to accelerate routine work, but never outsource your judgment.